

TUGAS 3
SISTEM OPERASI



Dosen Pembimbing:
Triawan Adi Cahyanto M.Kom

Dibuat oleh:
Darina Qothrun Nada [2410651047]

TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

Tugas 1: Kalkulator Paralel

Buat program yang:

1. Menerima input 2 angka dari user
2. Menghitung penjumlahan, pengurangan, perkalian, dan pembagian
3. Setiap operasi dikerjakan oleh thread berbeda
4. Tampilkan semua hasil setelah semua thread selesai

nano kalkulator_paralel.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
// Struktur untuk menampung data input dan hasil setiap operasi
```

```
typedef struct {
```

```
    double num1;
```

```
    double num2;
```

```
    double result;
```

```
    char operation;
```

```
} OperationData;
```

```
// Array global untuk menyimpan data operasi (agar mudah diakses oleh thread)
```

```
OperationData op_data[4];
```

```
// --- Fungsi Thread untuk Penjumlahan (Addition) ---
```

```
void *add_func(void *arg) {
```

```
    OperationData *data = (OperationData *)arg;
```

```
    data->result = data->num1 + data->num2;
```

```
    printf("[INFO] Thread Penjumlahan selesai.\n");
```

```
    pthread_exit(NULL);
```

```
}
```

```
// --- Fungsi Thread untuk Pengurangan (Subtraction) ---
```

```
void *sub_func(void *arg) {
```

```

    OperationData *data = (OperationData *)arg;
    data->result = data->num1 - data->num2;
    printf("[INFO] Thread Pengurangan selesai.\n");
    pthread_exit(NULL);
}

// --- Fungsi Thread untuk Perkalian (Multiplication) ---
void *mul_func(void *arg) {
    OperationData *data = (OperationData *)arg;
    data->result = data->num1 * data->num2;
    printf("[INFO] Thread Perkalian selesai.\n");
    pthread_exit(NULL);
}

// --- Fungsi Thread untuk Pembagian (Division) ---
void *div_func(void *arg) {
    OperationData *data = (OperationData *)arg;
    // Pengecekan pembagian dengan nol
    if (data->num2 == 0) {
        fprintf(stderr, "[ERROR] Pembagian dengan nol tidak diizinkan.\n");
        data->result = 0.0; // Atur hasil ke nol atau nilai error
    } else {
        data->result = data->num1 / data->num2;
    }
    printf("[INFO] Thread Pembagian selesai.\n");
    pthread_exit(NULL);
}

// --- Fungsi Utama (Main Function) ---
int main() {
    double a, b;
    pthread_t threads[4];

```

```

// 1. Menerima input 2 angka dari user
printf("=====\\n");
printf("    KALKULATOR PARALEL\\n");
printf("=====\\n");
printf("Masukkan angka pertama (A): ");
if (scanf("%lf", &a) != 1) {
    fprintf(stderr, "Input tidak valid.\\n");
    return 1;
}
printf("Masukkan angka kedua (B): ");
if (scanf("%lf", &b) != 1) {
    fprintf(stderr, "Input tidak valid.\\n");
    return 1;
}

// 2. Menginisialisasi data input untuk setiap operasi
// Indeks 0: Penjumlahan
op_data[0] = (OperationData){a, b, 0.0, '+'};
// Indeks 1: Pengurangan
op_data[1] = (OperationData){a, b, 0.0, '-'};
// Indeks 2: Perkalian
op_data[2] = (OperationData){a, b, 0.0, '*'};
// Indeks 3: Pembagian
op_data[3] = (OperationData){a, b, 0.0, '/'};

// 3. Membuat 4 thread, setiap thread mengerjakan operasi berbeda
printf("\\n--- Memulai Komputasi Paralel ---\\n");

pthread_create(&threads[0], NULL, add_func, (void *)&op_data[0]);
pthread_create(&threads[1], NULL, sub_func, (void *)&op_data[1]);
pthread_create(&threads[2], NULL, mul_func, (void *)&op_data[2]);

```

```

pthread_create(&threads[3], NULL, div_func, (void *)&op_data[3]);

// Menunggu (Join) semua thread selesai
for (int i = 0; i < 4; i++) {
    pthread_join(threads[i], NULL);
}

printf("--- Semua Thread Selesai ---\n");

// 4. Menampilkan semua hasil setelah semua thread selesai
printf("\n=====\\n");
printf("      HASIL AKHIR\\n");
printf("=====\\n");
printf("Penjumlahan (%.2lf %c %.2lf) = %.2lf\\n", a, op_data[0].operation, b, op_data[0].result);
printf("Pengurangan (%.2lf %c %.2lf) = %.2lf\\n", a, op_data[1].operation, b, op_data[1].result);
printf("Perkalian (%.2lf %c %.2lf) = %.2lf\\n", a, op_data[2].operation, b, op_data[2].result);
if (b == 0) {
    printf("Pembagian (%.2lf %c %.2lf) = Pembagian dengan nol (Error)\\n", a, op_data[3].operation,
b);
} else {
    printf("Pembagian (%.2lf %c %.2lf) = %.2lf\\n", a, op_data[3].operation, b, op_data[3].result);
}

return 0;
}

```

- Karena Anda menggunakan Pthreads, Anda harus menyertakan opsi library pthread saat mengompilasi.

Bash : gcc kalkulator_paralel.c -o kalkulator_paralel -pthread

Eksekusi: ./kalkulator_paralel

- **Konsep Utama:**

Tujuan utama dari tugas ini adalah membuktikan bahwa empat operasi matematika (tambah, kurang, kali, bagi) dapat berjalan secara simultan (paralel) dengan menugaskan setiap operasi ke thread yang berbeda.

Thread (Alur Eksekusi) : Merupakan bagian dari suatu proses. Dalam kasus ini, proses utama (main) membuat empat thread anak, yang memungkinkan CPU untuk mengerjakan keempat operasi secara bersamaan, menghemat waktu komputasi dibandingkan jika dikerjakan secara berurutan.

- **Struktur Data**

- Tujuan: Karena fungsi thread di Pthreads (`void *func(void *arg)`) hanya menerima satu argumen bertipe `void *`, kita perlu mengemas (bundling) semua data yang dibutuhkan (dua angka input dan tempat untuk hasil) ke dalam satu struktur.
- Akses Data: Setiap thread akan menerima pointer ke instansi `OperationData` miliknya sendiri, memastikan setiap operasi bekerja pada data yang terisolasi.

- **Fungsi Thread**

- Penerimaan Argumen: Di awal setiap fungsi, argumen `void *arg` (yang merupakan alamat dari struktur data) diubah kembali (di-casting) menjadi tipe `OperationData *data`.
- Pengerjaan: Masing-masing fungsi hanya melakukan satu operasi spesifik (misalnya, `add_func` hanya melakukan penjumlahan) dan menyimpan hasilnya kembali ke kolom result dari struktur data yang sama.
- `pthread_exit(NULL)`: Ini adalah cara standar bagi thread untuk mengakhiri eksekusinya dan mengembalikan kontrol.

- **Fungsi Utama**

- A. **Inisialisasi dan Pembuatan Thread (Poin 1, 2, 3)**

- `pthread_t threads[4]`: Mendeklarasikan array untuk menyimpan ID pengenalan (handle) dari keempat thread yang akan dibuat.
 - `pthread_create(...)`: Fungsi ini memulai thread baru :
 - Argumen 1 (`&threads[i]`): Tempat menyimpan ID thread baru.
 - Argumen 3 (`add_func`): Fungsi yang akan dijalankan oleh thread ini.
 - Argumen 4 (`((void *)&op_data[i])`): Argumen yang dilewatkan ke fungsi thread. Ini adalah pointer ke data operasi yang spesifik.

- B. **Sinkronisasi**

- `pthread_join`: Ini adalah bagian terpenting untuk sinkronisasi (memastikan urutan waktu). `pthread_join(threads[i], NULL)` memaksa program utama (main) untuk menunggu hingga thread dengan ID `threads[i]` selesai mengeksekusi.
 - Penting: Karena kita melakukan join untuk semua 4 thread sebelum menampilkan hasil, ini menjamin bahwa semua operasi telah selesai sebelum output dicetak, memenuhi poin terakhir tugas.

- C. **Penampilan Hasil**

- Setelah semua thread di-join, kita tahu bahwa kolom result di setiap elemen `op_data` sudah terisi dengan hasil perhitungan yang benar. Program utama kemudian hanya perlu mencetak hasilnya dari struktur data tersebut.

Tugas 2: File Processing

Buat program yang:

1. Membaca file teks
2. Menghitung jumlah kata menggunakan 2 thread (bagi file jadi 2 bagian)
3. Gabungkan hasilnya
4. Bandingkan waktu eksekusi dengan versi single-thread

nano sample_text.txt

```
import threading
import time
import re
from typing import Dict

# GANTI DENGAN NAMA FILE TEKS ANDA!
FILE_PATH = "sample_text.txt"
NUM_THREADS = 2

def single_thread_word_count(file_path: str) -> (Dict[str, int], float):
    """Menghitung kata dalam seluruh file menggunakan satu thread."""
    start_time = time.perf_counter()
    word_counts = {}

    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            content = f.read()
            words = re.findall(r'\b\w+\b', content.lower())

            for word in words:
                word_counts[word] = word_counts.get(word, 0) + 1

    except FileNotFoundError:
        print(f'Error: File tidak ditemukan di {file_path}')
        return {}, 0.0

    end_time = time.perf_counter()
    return word_counts, end_time - start_time

def read_file_chunk_and_count(file_path: str, start_byte: int, end_byte: int, results: Dict[str,
int], lock: threading.Lock):
    """Fungsi target untuk thread: Membaca sebagian file dan menghitung kata."""

    local_word_counts = {}

    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            f.seek(start_byte)
            chunk_content = f.read(end_byte - start_byte)
            words = re.findall(r'\b\w+\b', chunk_content.lower())
```

```

        for word in words:
            local_word_counts[word] = local_word_counts.get(word, 0) + 1

except FileNotFoundError:
    return

# Gabungkan hasilnya ke dictionary global menggunakan Lock
with lock:
    for word, count in local_word_counts.items():
        results[word] = results.get(word, 0) + count

def multi_thread_word_count(file_path: str, num_threads: int = 2) -> (Dict[str, int], float):
    """Menghitung kata menggunakan banyak thread."""
    start_time = time.perf_counter()

    try:
        with open(file_path, 'rb') as f:
            f.seek(0, 2)
            file_size = f.tell()
    except FileNotFoundError:
        return {}, 0.0

    chunk_size = file_size // num_threads
    threads = []
    final_counts = {}
    lock = threading.Lock()

    for i in range(num_threads):
        start_byte = i * chunk_size
        end_byte = (i + 1) * chunk_size if i < num_threads - 1 else file_size

        thread = threading.Thread(target=read_file_chunk_and_count,
                                  args=(file_path, start_byte, end_byte, final_counts, lock))
        threads.append(thread)
        thread.start()

    for thread in threads:
        thread.join()

    end_time = time.perf_counter()
    return final_counts, end_time - start_time

# --- EKSEKUSI UTAMA ---
if __name__ == "__main__":
    print(f"Proses File: *{FILE_PATH}*\n")

    # Versi Single-thread
    counts_single, time_single = single_thread_word_count(FILE_PATH)
    total_words_single = sum(counts_single.values())

```

```

# Versi Multi-thread (2 Thread)
counts_multi, time_multi = multi_thread_word_count(FILE_PATH,
num_threads=NUM_THREADS)
total_words_multi = sum(counts_multi.values())

print("==== Hasil Perbandingan Waktu ====")
print(f"Total Kata (Single-thread): {total_words_single}")
print(f"Total Kata (Multi-thread): {total_words_multi}")
print("-" * 36)
print(f"Waktu Eksekusi *Single-thread*: {time_single:.4f} detik")
print(f"Waktu Eksekusi *Multi-thread ({NUM_THREADS} T)*: {time_multi:.4f} detik")
print("-" * 36)

if time_single > 0 and total_words_single == total_words_multi:
    if time_multi < time_single:
        speedup = time_single / time_multi
        print(f"Kesimpulan: Multi-thread lebih cepat *{speedup:.2f}x* (Speedup). 🚀")
    elif time_multi > time_single:
        print(f"Kesimpulan: Single-thread lebih cepat (Kemungkinan karena Overhead Threading dan GIL). 🐢")
    else:
        print("Kesimpulan: Waktu eksekusi hampir sama.")
elif total_words_single != total_words_multi:
    print("Peringatan: Jumlah kata tidak sama! Periksa pembagian file.")

```

➤ Fungsi Utama Kode Penghitung Kata

Kode ini adalah eksperimen untuk membandingkan kecepatan menghitung kata dalam sebuah file teks menggunakan dua cara: Satu Thread (cara biasa) dan Banyak Thread (cara paralel).

1. Pengaturan Awal

- **Import:** Menggunakan `$\text{threading}$` untuk pekerjaan paralel, `$\text{time}$` untuk mengukur waktu, dan `$\text{re}$` (ekspresi reguler) untuk menemukan kata.
- **Konstanta:**
 - `$\text{FILE\_PATH}$`: File teks yang akan dihitung.
 - `$\text{NUM\_THREADS}$`: Jumlah pekerja (thread) yang akan digunakan untuk mode paralel.

2. Fungsi Satu Thread (`$\text{single\_thread\_word\_count}$`)

Ini adalah patokan kecepatan (baseline):

- **Pekerjaan Sekali Jalan:** Membaca seluruh konten file dari awal sampai akhir, hanya dengan satu prosesor.
- **Penghitungan:** Mencari dan menghitung frekuensi setiap kata (mengabaikan tanda baca).
- **Hasil:** Mengembalikan kamus (kata: jumlah) dan waktu total yang dibutuhkan.

3. Fungsi Banyak Thread (`$\text{multi\_thread\_word\_count}$`)

Ini adalah metode paralel (berpotensi lebih cepat):

- **Pembagian Tugas:** Kode menghitung ukuran file dan membaginya menjadi `$\text{NUM\_THREADS}$` bagian (chunk) yang sama besar (berdasarkan byte).
- **Pekerja Paralel:** Untuk setiap bagian, dibuat satu thread baru.

- Setiap thread hanya membaca dan menghitung kata pada chunk yang menjadi jatahnya.
- Menggabungkan Hasil (Lock): Ketika thread selesai, ia harus menggabungkan hitungannya ke dalam kamus total (final_counts).
 - Ia menggunakan threading.Lock (seperti kunci pintu) untuk memastikan hanya satu thread yang bisa menulis ke kamus tersebut pada saat bersamaan. Ini mencegah hasil hitungan menjadi kacau (race condition).
- Sinkronisasi: Kode utama menunggu sampai semua thread selesai (thread.join()) sebelum menghitung total waktu.

4. Hasil dan Kesimpulan

- Kode membandingkan waktu time_single vs. time_multi .
- Multi-thread Menang (🏆): Jika time_multi lebih kecil, kode menghitung rasio peningkatan kecepatan (speedup).
- Single-thread Menang (🏆): Jika time_single lebih kecil, ini sering disebabkan oleh:
 - Overhead Threading: Butuh waktu untuk membuat dan mengelola thread baru.
 - GIL (Global Interpreter Lock): Di Python, GIL membatasi CPU untuk hanya menjalankan satu thread pada satu waktu. Jadi, proses yang banyak melibatkan CPU (seperti menghitung kata) seringkali tidak mendapat manfaat dari multi-threading.
- Peringatan Penting: Peringatan "Jumlah kata tidak sama!" muncul jika pembagian chunk memotong kata menjadi dua. Ini menyebabkan beberapa kata terlewat dan hasilnya tidak akurat.

Tugas 3: Producer-Consumer

1. Buat simulasi producer-consumer dengan:
2. Thread producer yang menghasilkan angka random
3. Thread consumer yang memproses angka tersebut
4. Gunakan mutex untuk sinkronisasi
5. Buffer maksimal 10 item

nano producer_consumer.py

```
import threading
import time
import random
from typing import List

# --- KONSTANTA GLOBAL ---
BUFFER_SIZE: int = 10 # Kapasitas maksimal buffer
SIMULATION_DURATION: int = 15 # Durasi simulasi dalam detik

# --- SUMBER DAYA BERSAMA ---
buffer: List[int] = []
# Condition object menggabungkan Lock (mutex) dan kemampuan untuk menunggu/memberi
sinyal
condition = threading.Condition()

class Producer(threading.Thread):
    """Thread yang menghasilkan angka random dan menaruhnya di buffer."""
    def __init__(self, name: str):
        super().__init__(name=name)

    def run(self):
        start_time = time.time()
        while (time.time() - start_time) < SIMULATION_DURATION:
            item = random.randint(1, 100)

            # --- KRITIS: MEMASUKKAN ITEM ---
            with condition:
                # 1. Producer menunggu jika buffer PENUH
                while len(buffer) == BUFFER_SIZE:
                    print(f"[{self.name}] Buffer Penuh. Menunggu...")
                    condition.wait() # Melepaskan mutex dan menidurkan thread

                # 2. Tambahkan item
                buffer.append(item)
                print(f"[{self.name}] Menghasilkan {item}. Ukuran buffer: {len(buffer)}")

                # 3. Beri sinyal ke Consumer bahwa ada item baru
```

```

        condition.notify()

        time.sleep(random.uniform(0.1, 0.5)) # Simulasi waktu produksi

class Consumer(threading.Thread):
    """Thread yang mengambil angka dari buffer dan memprosesnya."""
    def __init__(self, name: str):
        super().__init__(name=name)

    def run(self):
        start_time = time.time()
        while (time.time() - start_time) < SIMULATION_DURATION:

            # --- KRITIS: MENGAMBIL ITEM ---
            with condition:
                # 1. Consumer menunggu jika buffer KOSONG
                while len(buffer) == 0:
                    print(f"[{self.name}] Buffer Kosong. Menunggu...")
                    condition.wait() # Melepaskan mutex dan menidurkan thread

                # 2. Ambil item
                item = buffer.pop(0)
                print(f"[{self.name}] Memproses {item} → Selesai. Ukuran buffer: {len(buffer)}")

                # 3. Beri sinyal ke Producer bahwa ada ruang kosong
                condition.notify()

            time.sleep(random.uniform(0.3, 1.0)) # Simulasi waktu pemrosesan

# --- EKSEKUSI UTAMA ---
if __name__ == "__main__":
    print(f"🌀 Memulai simulasi Producer-Consumer dengan Buffer Max: {BUFFER_SIZE}")

    producer_thread = Producer("PRODUCER")
    consumer_thread = Consumer("CONSUMER")

    producer_thread.start()
    consumer_thread.start()

    producer_thread.join()
    consumer_thread.join()

    print("\n✅ Simulasi Selesai.")
    print(f"Item tersisa dalam buffer: {buffer}")

```

1. Peran Para Pemain (Thread)

- Producer (Pembuat Angka): Ini adalah orang yang selalu ingin menaruh barang (angka acak) ke dalam loker.
- Consumer (Pemroses Angka): Ini adalah orang yang selalu ingin mengambil barang dari loker untuk diproses.
- Buffer (Loker): Ini adalah tempat penyimpanan bersama, dibatasi maksimal 10 item.

2. Aturan Kunci (Sinkronisasi dengan `\text{Condition}` / `Mutex`)

Kita menggunakan Petugas Penjaga Kunci (`Mutex`) untuk memastikan loker aman. Di kode, ini diwakili oleh `\text{with condition:}`.

▪ Aturan 1: Loker Penuh!

- Jika Producer datang dan loker sudah penuh (10 item), Petugas Penjaga akan menyuruhnya TIDUR (`\text{wait()}`).
- Producer akan tidur dan melepaskan kuncinya, menunggu sinyal dari Consumer.

▪ Aturan 2: Loker Kosong!

- Jika Consumer datang dan loker kosong (0 item), Petugas Penjaga akan menyuruhnya TIDUR (`\text{wait()}`).
- Consumer akan tidur dan melepaskan kuncinya, menunggu sinyal dari Producer.

• Aturan 3: Saling Beri Sinyal (`\text{notify()}`):

- Ketika Producer berhasil menaruh barang, ia akan membangunkan Consumer (karena sekarang ada barang untuk diambil).
- Ketika Consumer berhasil mengambil barang, ia akan membangunkan Producer (karena sekarang ada ruang kosong di loker).